

Kanbien Front-End Architecture and Design System Guide

React, enterprise design systems, token architecture, and a worked Button
example

June 21, 2026

Table of Contents

Executive framing	4
Part 1: React from first principles	5
What React is	5
What React is not	5
Why React became dominant	6
Component-based development maps well to product teams	6
React made UI more declarative	6
React made reuse practical	7
React is flexible rather than fully prescriptive	7
The ecosystem became massive	7
What problems React solves compared to plain HTML, CSS, and JavaScript	7
Problem 1: Keeping UI synchronized with state	7
Problem 2: Reusing UI safely	8
Problem 3: Managing complex interaction	8
Problem 4: Building large applications with many teams	9
Tradeoffs React introduces	9
Tradeoff 1: More tooling	9
Tradeoff 2: Learning curve	9
Tradeoff 3: Too much architectural freedom	9
Tradeoff 4: Performance mistakes are easy	10
Tradeoff 5: Accessibility is not automatic	10
When React is the wrong choice	10
How React differs from Angular, Vue, Svelte, and Web Components	10
Why most enterprise software teams choose React today	11
Part 2: Design systems from first principles	13
What a design system actually is	13
The design-system layer model	13
Design tokens	14
Foundation tokens	15
Semantic tokens	16
Builder tokens	16
Primitives	17
Patterns	18
Styling and behavior patterns	18
UX and product patterns	18
Components	19
Templates	19
Pages	20
How the layers depend on each other	20
Which layers should be reusable across products?	21
Which layers should be application-specific?	21
Common mistakes teams make when building design systems	22
Mistake 1: Starting with components before tokens	22
Mistake 2: Letting applications use raw foundation tokens	22
Mistake 3: Creating too many tokens too early	22
Mistake 4: Confusing Figma assets with a design system	23
Mistake 5: Ignoring accessibility	23
Mistake 6: Overloading variants	23
Mistake 7: Allowing business logic into design-system components	23
Mistake 8: Theming with React props everywhere	23
Mistake 9: No governance model	24
Part 3: Design-system package structure in a TypeScript monorepo	25
tokens/	27
Purpose	27
Contents	27

Example files	27
Example foundation token	27
Allowed dependencies	28
Forbidden dependencies	28
Ownership rules	28
resolvers/	28
Purpose	28
Contents	29
Example resolver	29
Allowed dependencies	29
Forbidden dependencies	29
Ownership rules	30
primitives/	30
Purpose	30
Contents	30
Example primitive	30
Allowed dependencies	30
Forbidden dependencies	31
Ownership rules	31
patterns/	31
Purpose	31
Contents	31
Example pattern	31
Allowed dependencies	32
Forbidden dependencies	32
Ownership rules	32
components/	32
Purpose	32
Contents	33
Example files	33
Allowed dependencies	33
Forbidden dependencies	33
Ownership rules	33
Recommended dependency rules	34
Part 4: Evaluating the design-label architecture	35
Is this a good architecture?	35
What large companies do that is similar	36
Potential pitfalls	36
Pitfall 1: Design labels become hidden product forks	36
Pitfall 2: Layout shifts across design labels	36
Pitfall 3: Accessibility regressions	37
Pitfall 4: Token alias cycles	37
Pitfall 5: Runtime token loading creates flicker	37
Pitfall 6: Third-party components ignore your tokens	37
Pitfall 7: Too many design labels	37
How to implement this cleanly in React	38
Part 5: Complete worked example - Button	40
Step 1: Design Label	40
Step 2: Foundation Tokens	40
Step 3: Semantic Tokens	43
Step 4: Builder Tokens	44
Step 5: Primitive	46
Step 6: Pattern	46
Step 7: Component	48
Step 8: Application Usage	49
Part 6: Final recommended Kanbien architecture	51

Executive framing

For Kanbien, think of the front end as two separate but connected systems:

Application system

Products, workflows, routes, data, permissions, business behavior

Design system

Visual language, reusable UI building blocks, accessibility rules, interaction rules, theming

The mistake many companies make is treating the design system as “a folder of React components.” That is too shallow. A serious design system is closer to **UI infrastructure**: it gives every product team a governed way to build interfaces without re-solving color, spacing, typography, accessibility, interaction states, and layout every sprint.

Part 1: React from first principles

What React is

React is a JavaScript library for building user interfaces. The official React documentation describes it as “the library for web and native user interfaces” and explains that it lets teams build UIs from reusable pieces called components: react.dev.

A React application is made from **components**.

A component is usually a JavaScript or TypeScript function that receives input and returns what should appear on the screen.

```
function Greeting({ name }: { name: string }) {  
  return <h1>Hello, {name}</h1>;  
}
```

That HTML-looking syntax is called **JSX**. JSX is not plain HTML. It is syntax that lets developers describe UI structure inside JavaScript or TypeScript. React’s docs describe JSX as a JavaScript syntax extension popularized by React: react.dev.

A simplified React mental model looks like this:

```
User clicks something  
      v  
Component state changes  
      v  
React re-runs affected component functions  
      v  
Components return new JSX  
      v  
React updates the browser DOM efficiently  
      v  
User sees the updated UI
```

In plain English, React lets engineers say:

“Given this data and this state, this is what the UI should look like.”

Instead of saying:

“Find this DOM node, remove this class, insert this div, update this text, add this event listener, and remember to clean it up later.”

That difference matters enormously once an application becomes interactive.

What React is not

React is not a backend framework.

React is not a database layer.

React is not an API framework.

React is not a design system.

React is not, by itself, a full enterprise application platform.

React is primarily the **UI rendering and component composition layer**. For real products, teams usually add routing, data fetching, state management, build tooling, testing, authentication integration, accessibility practices, deployment pipelines, and observability around it.

That is why a production Kanbien front end should be thought of as:

```
React  
+ TypeScript  
+ routing
```

- + data access
- + design system
- + build tooling
- + testing
- + accessibility
- + CI/CD
- + governance

React is the engine for rendering UI, not the entire vehicle.

Why React became dominant

React became dominant because it fit the shape of modern product development.

Enterprise software changed from mostly static pages to highly interactive applications: dashboards, workflows, forms, modals, side panels, data grids, notifications, collaborative states, permissions, saved filters, search, inline editing, live validation, and personalization.

Plain HTML, CSS, and JavaScript can do all of that, but doing it at scale becomes hard because teams end up manually coordinating UI state and DOM updates.

React's popularity comes from several reinforcing advantages.

Component-based development maps well to product teams

React lets teams build user interfaces out of independent components, then combine those components into screens and applications. React's docs emphasize this compositional model and note that components written by different people, teams, and organizations can be combined: react.dev.

That maps very well to enterprise delivery:

```
Design system team owns:
Button, Input, Modal, Tooltip, Tabs, Table, DatePicker

Product team owns:
CustomerSearchPage, InvoiceApprovalFlow, RiskDashboard

Platform team owns:
routing, authentication, observability, build tooling
```

Component boundaries create organizational boundaries.

React made UI more declarative

Without React, developers often write imperative DOM manipulation:

```
const button = document.querySelector("#save-button");
button.textContent = "Saving...";
button.disabled = true;
button.classList.add("loading");
```

In React, the UI is derived from state:

```
function SaveButton({ isSaving }: { isSaving: boolean }) {
  return (
    <button disabled={isSaving}>
      {isSaving ? "Saving..." : "Save"}
    </button>
  );
}
```

The React version is easier to reason about because the component describes the desired UI for each state.

React made reuse practical

Before component frameworks, teams often reused CSS classes, snippets, templates, or jQuery plugins. That helped, but it did not fully package structure, behavior, state, accessibility, and styling into one reusable unit.

A proper React component can contain:

```
DOM structure
Props API
State behavior
Events
Accessibility attributes
Styling hooks
Tests
Documentation
Design-token usage
```

That makes React a natural fit for design systems.

React is flexible rather than fully prescriptive

Angular gives teams a large first-party application framework. React gives teams a UI library that can be combined with different routers, build tools, data-fetching strategies, and rendering frameworks.

That flexibility helped React spread because teams could adopt it incrementally rather than buying into one complete framework philosophy.

The ecosystem became massive

React's ecosystem includes component libraries, accessibility libraries, form libraries, data-fetching tools, testing tools, full-stack frameworks, design-system tooling, mobile options through React Native, and a large hiring market.

The 2025 Stack Overflow Developer Survey reported React at 44.7% among all respondents and 46.9% among professional developers for web frameworks and web technologies, ahead of Angular, Vue, and Svelte in that survey's usage data: survey.stackoverflow.co.

That matters in enterprise technology selection because the best architecture is not only technically elegant. It must also be staffable, maintainable, supportable, and easy to govern.

What problems React solves compared to plain HTML, CSS, and JavaScript

Plain HTML, CSS, and JavaScript are the foundation of the web. React does not replace them. React organizes them.

Problem 1: Keeping UI synchronized with state

Imagine a button with these states:

```
default
hover
focus
loading
disabled
success
error
permission denied
```

In plain JavaScript, developers often update the DOM manually whenever state changes. With React, the UI is a function of state.

```
function SubmitButton({
  status,
}): {
  status: "idle" | "loading" | "success" | "error";
}) {
  return (
    <button disabled={status === "loading"}>
      {status === "loading" ? "Submitting..." : "Submit"}
    </button>
  );
}
```

The code says what the button should look like for a given state. React handles updating the browser.

Problem 2: Reusing UI safely

Without a component model, teams often copy markup:

```
<button class="primary-button">
  Save
</button>
```

Then another team tweaks it:

```
<button class="primary-button custom-save-button">
  Save changes
</button>
```

Over time, small differences accumulate. Accessibility breaks. Spacing drifts. Visual language fragments.

React encourages reusable components:

```
<Button variant="primary">Save changes</Button>
```

The consuming team does not need to remember the correct class names, ARIA attributes, focus behavior, loading behavior, or icon spacing. The design system owns that.

Problem 3: Managing complex interaction

Modern enterprise UI is not just pages. It includes modals, popovers, menus, comboboxes, drag-and-drop, keyboard navigation, data tables, optimistic updates, validation, pagination, and permissions.

React's component model lets engineers isolate those concerns.

```
DataTable
|-- TableToolbar
|-- ColumnVisibilityMenu
|-- FilterPanel
|-- Pagination
`-- EmptyState
```

Each part has a clear owner and test surface.

Problem 4: Building large applications with many teams

React makes UI modular. A design system can ship a Button, Modal, TextField, and Toast. Product teams can compose those into workflows. Platform teams can govern the package through versioning, linting, Storybook, tests, and changelogs.

That is hard to enforce if every application writes raw HTML and CSS freely.

Tradeoffs React introduces

React solves many problems, but it is not free.

Tradeoff 1: More tooling

A serious React application usually needs:

```
TypeScript
bundler/build tool
package manager
test runner
linting
formatting
component documentation
CSS strategy
routing
data-fetching strategy
accessibility tooling
visual regression testing
```

For a small static website, that can be overkill.

Tradeoff 2: Learning curve

A new React developer must understand:

```
components
props
state
hooks
effects
JSX
rendering
composition
controlled vs uncontrolled inputs
memoization
server/client rendering boundaries in modern frameworks
```

React Hooks let components use React features like state and context, and React's docs describe state as what lets a component "remember" information such as user input: react.dev.

For a non-front-end team, that conceptual model takes time.

Tradeoff 3: Too much architectural freedom

React's flexibility is a strength and a weakness. Without strong front-end architecture, teams can create inconsistent patterns for:

```
state management
forms
API calls
CSS
routing
error handling
```

```
loading states
authorization checks
component APIs
```

Angular prevents some of this by being more opinionated. React requires stronger engineering governance.

Tradeoff 4: Performance mistakes are easy

React is fast enough for most software, but teams can still create performance problems by:

```
re-rendering huge component trees unnecessarily
passing unstable objects/functions
loading too much JavaScript
using expensive client-side rendering where server rendering would be better
building large component libraries without tree-shaking
```

React will not save a poorly designed front-end architecture.

Tradeoff 5: Accessibility is not automatic

A React component can be accessible or inaccessible. React does not magically fix poor semantics.

For example, a real `<button>` should generally be used instead of a clickable `<div role="button">`; MDN notes that adding `role="button"` does not provide native button behavior like keyboard handling, and generally recommends using `<button>` or `<input type="button">` instead: developer.mozilla.org.

This is exactly why a design system is valuable: it centralizes accessibility decisions.

When React is the wrong choice

React is the wrong choice when the product does not need React's complexity.

Use plain HTML/CSS/JavaScript, server-rendered templates, or a lighter approach when:

```
The site is mostly static content.
The interaction model is simple.
The team has no React capability and little need for it.
The business needs fast content publishing more than rich application behavior.
The UI is a small embedded widget that must run in any host page.
The organization already has a strong standard around another framework.
The app is mostly server-driven forms with minimal dynamic behavior.
```

React may also be the wrong choice when the team wants a fully prescribed framework from day one. Angular can be better for organizations that explicitly prefer one integrated, opinionated platform.

For Kanbien, because you are designing a greenfield platform and care about a reusable design system, theming, product consistency, and multiple future applications, React is a reasonable default. But the decision should be paired with strong architecture. React without front-end governance becomes a mess quickly.

How React differs from Angular, Vue, Svelte, and Web Components

Technology	What it is	Main philosophy	Strength	Tradeoff
React	UI library	Components + declarative rendering + ecosystem choice	Flexible, huge ecosystem, strong design-system fit	Requires architectural decisions around routing, data, state, CSS
Angular	Full web framework	Opinionated application platform	Built-in routing, forms, dependency injection, conventions	Heavier learning curve, more framework commitment
Vue	Progressive framework	Incrementally adoptable framework	Approachable, flexible, good single-file component model	Smaller enterprise ecosystem than React in many markets
Svelte	Compiler-based UI framework	Compile components to efficient JavaScript	Concise code, less runtime work	Smaller ecosystem and hiring pool than React
Web Components	Browser-native component standards	Custom elements, Shadow DOM, templates	Framework-agnostic distribution	Lower-level developer experience; design-system ergonomics can be harder

Angular’s own docs describe it as a web framework maintained by Google, with a broad suite of tools, APIs, and libraries, including components, dependency injection, routing, and forms: angular.dev.

Vue describes itself as “The Progressive Framework” and emphasizes flexibility and incremental adoption, including static HTML enhancement, SPA usage, SSR, SSG, and more: vuejs.org.

Svelte is different because it uses a compiler; its docs say Svelte lets developers write concise components that do minimal work in the browser, and SvelteKit’s docs explain that the compiler converts components into JavaScript and CSS for the page: svelte.dev.

Web Components are not a framework. They are browser technologies: custom elements, Shadow DOM, and HTML templates. MDN describes them as technologies for creating reusable custom elements with encapsulated functionality and reduced collision risk: developer.mozilla.org.

Why most enterprise software teams choose React today

Enterprise teams tend to choose React because it optimizes for the realities of large-scale product engineering:

```

large hiring pool
many libraries
strong TypeScript support
strong design-system fit
large community
good testing ecosystem
usable with many rendering strategies
compatible with incremental adoption
many engineers already know it

```

The strategic reason is not that React is always technically superior. The strategic reason is that React has become a **safe enterprise default**: it is staffable, well-supported, flexible, and proven across many product categories.

For Kanbien, that matters. A greenfield platform should not optimize only for elegance. It should optimize for:

```
delivery speed
governance
maintainability
accessibility
theming
hiring
reuse
future product expansion
```

React is a strong choice when paired with a disciplined design system.

Part 2: Design systems from first principles

What a design system actually is

A design system is not a style guide.

A design system is not a Figma file.

A design system is not a React component library.

A design system is the governed system of decisions, assets, rules, code, documentation, and processes that allow teams to build consistent user interfaces at scale.

A serious design system contains:

```
Design principles
Brand rules
Accessibility rules
Design tokens
Typography
Color
Spacing
Iconography
Motion
Layout rules
Interaction rules
Reusable primitives
Reusable components
Reusable patterns
Documentation
Figma libraries
Code packages
Testing standards
Contribution process
Versioning and deprecation policy
Governance model
```

IBM's Carbon describes itself as an open-source design system consisting of working code, design tools and resources, human interface guidelines, and a contributor community: carbon.designsystem.com.

That is the right scale of thinking.

For Kanbien, your design system should be treated as a **platform capability**, not a UI side project.

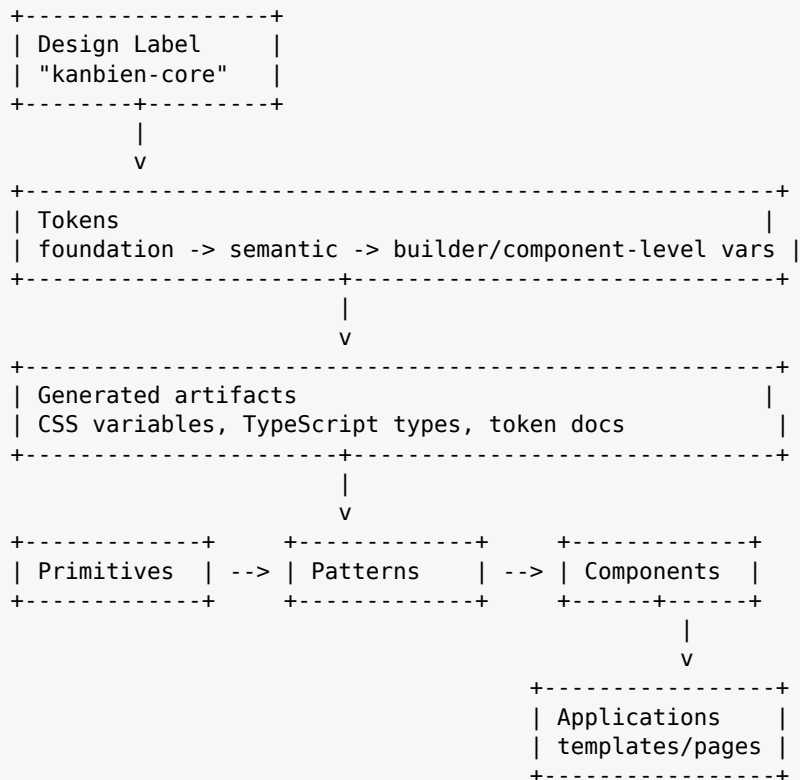
The design-system layer model

Here is the layered model I recommend for Kanbien:

```
Design Label
  v
Foundation Tokens
  v
Semantic Tokens
  v
Builder Tokens
  v
Primitives
  v
Patterns
  v
Components
  v
Templates
  v
```

Pages

A more technical dependency diagram:



Design tokens

Design tokens are named design decisions stored as data.

The Design Tokens Community Group describes design tokens as a methodology for expressing design decisions in a platform-agnostic way, creating a common vocabulary across tools and technologies: designtokens.org.

The same specification describes a token as information associated with a human-readable name, at minimum a name/value pair: designtokens.org.

Example:

```

{
  "color.brand.primary.600": {
    "$type": "color",
    "$value": "#1457D9"
  }
}

```

Instead of this:

```

.button {
  background: #1457D9;
}

```

You want this:

```

.button {
  background: var(--kb-color-action-primary-bg);
}

```

The token name carries meaning. The hex code does not.

Foundation tokens

Foundation tokens are the raw ingredients of the visual language.

They are usually close to the brand or visual identity:

```
color palettes
font families
font sizes
font weights
line heights
spacing scale
radius scale
shadow scale
motion durations
motion easing
z-index scale
breakpoints
```

Example:

```
{
  "color.blue.600": {
    "$type": "color",
    "$value": "#1457D9"
  },
  "space.4": {
    "$type": "dimension",
    "$value": "16px"
  },
  "radius.md": {
    "$type": "dimension",
    "$value": "8px"
  }
}
```

Foundation tokens should not normally be used directly by product components.

Bad:

```
.button {
  background: var(--kb-color-blue-600);
}
```

Better:

```
.button {
  background: var(--kb-color-action-primary-bg);
}
```

Why? Because `blue-600` describes what the color is. `action-primary-bg` describes what the color does.

Figma describes primitive tokens similarly as raw foundational values without context, while semantic tokens give raw values a role in the interface: [figma.com](https://www.figma.com/).

In Kanbien terminology, I would use:

```
Foundation tokens = raw system values
Semantic tokens   = role-based decisions
Builder tokens    = component/pattern construction decisions
```

Semantic tokens

Semantic tokens name the **purpose** of a value.

Examples:

```
{
  "color.surface.canvas": {
    "$type": "color",
    "$value": "{color.neutral.0}"
  },
  "color.text.default": {
    "$type": "color",
    "$value": "{color.neutral.950}"
  },
  "color.action.primary.bg": {
    "$type": "color",
    "$value": "{color.blue.600}"
  },
  "color.action.primary.text": {
    "$type": "color",
    "$value": "{color.neutral.0}"
  }
}
```

Semantic tokens answer:

What is this value for?

Not:

What is this value?

Good semantic token names:

```
color.text.default
color.text.subtle
color.surface.canvas
color.surface.raised
color.border.default
color.border.focus
color.action.primary.bg
color.action.primary.bg.hover
color.action.danger.bg
```

Bad semantic token names:

```
color.blueButton
color.grayText
color.bigPadding
color.marketingPurple
```

The goal is for the same semantic token to survive visual redesigns.

color.action.primary.bg

could be blue in one design label, green in another, purple in another, and still mean “background for the primary action.”

Builder tokens

“Builder tokens” is not as universally standardized a term as design tokens or semantic tokens. For Kanbien, I would define builder tokens as:

Component- and pattern-level implementation tokens used to construct reusable UI. They sit between semantic design decisions and actual component CSS.

Example:

```
{
  "button.primary.bg.default": {
    "$type": "color",
    "$value": "{color.action.primary.bg}"
  },
  "button.primary.bg.hover": {
    "$type": "color",
    "$value": "{color.action.primary.bg.hover}"
  },
  "button.primary.text": {
    "$type": "color",
    "$value": "{color.action.primary.text}"
  },
  "button.radius": {
    "$type": "dimension",
    "$value": "{radius.control}"
  },
  "button.height.md": {
    "$type": "dimension",
    "$value": "{size.control.md}"
  }
}
```

These are sometimes called **component tokens** in other systems. Carbon documents component tokens as tokens specific to a component and says they should not be used outside their own component: carbondesignsystem.com.

For Kanbien, builder tokens should usually be private to the design system package.

Application teams should use:

```
<Button variant="primary" />
```

They should not reach into:

```
--kb-button-primary-bg-default
```

unless you intentionally expose a controlled customization surface.

Primitives

Primitives are low-level UI building blocks.

They are not product-specific. They are not workflow-specific. They are small, reusable, and boring.

Examples:

```
Box
Text
Stack
Inline
Grid
Icon
VisuallyHidden
FocusRing
ButtonPrimitive
Slot
```

A primitive should not know what an “invoice,” “project,” “customer,” “risk,” or “approval” is.

Example:

```
<Box padding="4" background="surface.canvas">
  <Text tone="default">Hello</Text>
</Box>
```

In practice, I would be careful with too many public primitives. Primitives are powerful but can also let application teams bypass your design system and create inconsistent UI.

A good rule:

```
Expose primitives for layout and accessibility.
Use components for user-facing controls.
```

Patterns

Patterns are reusable solutions to recurring interface problems.

There are two types.

Styling and behavior patterns

These are implementation recipes used by components.

Examples:

```
button pattern
field pattern
focus ring pattern
surface pattern
density pattern
loading pattern
```

UX and product patterns

These are repeatable user-experience structures.

Examples:

```
empty state
error state
confirmation dialog
filter toolbar
bulk action bar
settings panel
wizard flow
search results layout
```

For the packages/design-system/patterns/ folder you proposed, I would mostly put **implementation patterns** and **domain-neutral UX patterns** there.

Do not put highly domain-specific workflows there.

Good design-system pattern:

```
EmptyState
```

Bad design-system pattern:

```
InvoiceDisputeResolutionEmptyState
```

That belongs to an application.

Components

Components are reusable, documented, tested UI controls with stable APIs.

Examples:

```
Button
TextField
Select
Checkbox
RadioGroup
Modal
Popover
Tabs
Toast
Tooltip
Table
Pagination
DatePicker
```

A component should package:

```
structure
style
states
accessibility
keyboard behavior
interaction behavior
variants
tests
documentation
design token usage
```

Example public API:

```
<Button variant="primary" size="md" loading>
  Save changes
</Button>
```

The consuming application should not care whether the button uses three spans, CSS variables, a spinner primitive, and six token layers internally.

The component API should express product intent.

Templates

Templates are reusable page-level structures.

Examples:

```
ListPageTemplate
DetailPageTemplate
SettingsPageTemplate
DashboardTemplate
AuthenticationLayout
AdminShell
SplitPaneTemplate
```

Templates are more dangerous to centralize than components because they get close to product behavior.

A design system may provide generic templates, but applications usually need to own concrete page composition.

Good shared template:

```
<ListPageTemplate
  title="Users"
  toolbar={<UserToolbar />}
  table={<UserTable />}
/>
```

Bad shared template:

```
<CustomerCreditRiskReviewPage />
```

That is a page, not a design-system template.

Pages

Pages are routed application screens.

Examples:

```
/users
/users/:id
/settings/billing
/projects/:projectId/risks
```

Pages own:

```
route parameters
data fetching
permissions
analytics
business rules
workflow orchestration
error boundaries
application-specific state
```

Pages should consume design-system components, not contain design-system logic.

How the layers depend on each other

The dependency direction should be one-way:

```
Foundation Tokens
  v
Semantic Tokens
  v
Builder Tokens
  v
Primitives
  v
Patterns
  v
Components
  v
Templates
  v
Pages
```

A lower layer must not depend on a higher layer.

Bad:

```
Button imports CustomerApprovalStatus
```

Good:

```
CustomerApprovalStatus uses Button
```

Bad:

```
tokens/foundation imports React
```

Good:

```
React components consume generated CSS variables from tokens
```

Bad:

```
Button checks current route and changes style
```

Good:

```
<Button variant="danger">Delete customer</Button>
```

The application decides intent. The design system decides presentation.

Which layers should be reusable across products?

For Kanbien, I would classify the layers like this:

Layer	Reusable across products?	Notes
Design label model	Yes	Shared concept: kanbien-core, kanbien-enterprise, etc. The structure is reusable; values may differ by label. Reusable across Kanbien family; partner/white-label products may override.
Foundation token schema	Yes	
Foundation token values	Usually	
Semantic token taxonomy	Yes	Critical to consistency. Keep stable.
Builder token taxonomy	Mostly yes	Usually private to components/patterns.
Primitives	Yes	Keep domain-neutral.
Patterns	Some	Domain-neutral patterns yes; workflow-specific patterns no.
Components	Yes	Core design-system value.
Templates	Partly	Generic templates yes; product-specific templates no.
Pages	No	Application-owned.

Which layers should be application-specific?

Application-specific layers should include:

```
routes
pages
business workflows
```

```
data fetching
authorization logic
analytics events
copy/content decisions
domain-specific components
domain-specific templates
application-specific feature flags
```

Applications may also define **application extension tokens**, but those should map back to semantic tokens wherever possible.

Example:

```
{
  "color.risk.high.bg": {
    "$type": "color",
    "$value": "{color.status.danger.bg}"
  }
}
```

That lets a risk product express domain meaning while still staying attached to the design system.

Common mistakes teams make when building design systems

Mistake 1: Starting with components before tokens

Teams build Button, Input, and Modal first, then later discover inconsistent colors, spacing, radius, and typography.

Better:

```
Define token taxonomy first.
Then build components from tokens.
```

Mistake 2: Letting applications use raw foundation tokens

If product teams use `blue.600` everywhere, a redesign becomes painful.

Use semantic intent:

```
color.action.primary.bg
color.text.default
color.border.focus
```

Mistake 3: Creating too many tokens too early

A token for every pixel becomes impossible to govern.

Bad:

```
card-header-title-left-padding-for-admin-dashboard
```

Better:

```
space.container.inline
space.stack.md
card.padding.inline
```

Mistake 4: Confusing Figma assets with a design system

A Figma library is one artifact. A React package is another artifact. The design system is the governed relationship between them.

Mistake 5: Ignoring accessibility

If the design system does not own accessibility, every product team will solve it inconsistently. The design system should own:

```
focus states
keyboard behavior
ARIA patterns
color contrast
reduced motion
screen reader behavior
disabled/loading states
```

Mistake 6: Overloading variants

A Button with 45 variants is not flexible. It is uncontrolled complexity.

Prefer small, intentional APIs:

```
<Button variant="primary" size="md" tone="neutral" />
<Button variant="secondary" size="sm" tone="danger" />
```

Avoid:

```
<Button
  blue
  rounded
  shadowed
  thick
  marketing
  compactButNotTooCompact
/>
```

Mistake 7: Allowing business logic into design-system components

This is a serious enterprise mistake.

Bad:

```
<Button showOnlyForAdminUsers />
```

Good:

```
{canDeleteUser ? <Button variant="danger">Delete</Button> : null}
```

Mistake 8: Theming with React props everywhere

Do not pass colors through React component props across the entire tree.

Bad:

```
<Button backgroundColor={theme.colors.primary600} />
```

Better:

```
<DesignProvider design="kanbien-core">  
  <Button variant="primary" />  
</DesignProvider>
```

Then the button uses CSS variables.

Mistake 9: No governance model

A design system without ownership becomes a dumping ground.

You need:

```
owners  
review process  
versioning  
breaking-change policy  
contribution process  
deprecation policy  
accessibility standards  
visual regression tests  
documentation requirements
```

Part 3: Design-system package structure in a TypeScript monorepo

You proposed:

```
packages/design-system/  
  tokens/  
  resolvers/  
  primitives/  
  patterns/  
  components/
```

That is a good high-level shape.

I would structure it like this:

```
packages/design-system/  
  package.json  
  tsconfig.json  
  README.md  
  
  tokens/  
    foundation/  
      kanbien-core.tokens.json  
      kanbien-compact.tokens.json  
      partner-acme.tokens.json  
  
    semantic/  
      color.semantic.tokens.json  
      space.semantic.tokens.json  
      typography.semantic.tokens.json  
      radius.semantic.tokens.json  
      motion.semantic.tokens.json  
  
    builder/  
      button.builder.tokens.json  
      field.builder.tokens.json  
      surface.builder.tokens.json  
      focus.builder.tokens.json  
  
    schema/  
      design-label.schema.ts  
      token.schema.ts  
  
    generated/  
      tokens.css  
      tokens.ts  
      token-names.ts  
      token-docs.json  
  
  resolvers/  
    resolveDesignLabel.ts  
    resolveTokenAliases.ts  
    createCssVariables.ts  
    validateContrast.ts  
    validateTokenGraph.ts  
    buildTokens.ts  
    __tests__/  
      resolveDesignLabel.test.ts  
      validateContrast.test.ts  
  
  primitives/  
    Box/  
      Box.tsx
```

```
Box.css
index.ts

Text/
  Text.tsx
  Text.css
  index.ts

Stack/
  Stack.tsx
  Stack.css
  index.ts

VisuallyHidden/
  VisuallyHidden.tsx
  index.ts

ButtonPrimitive/
  ButtonPrimitive.tsx
  index.ts

patterns/
  focusRing/
    focusRing.css
    focusRing.ts

  button/
    button.pattern.ts
    button.pattern.css
    index.ts

  field/
    field.pattern.ts
    field.pattern.css
    index.ts

  surface/
    surface.pattern.ts
    surface.pattern.css
    index.ts

components/
  Button/
    Button.tsx
    Button.css
    Button.test.tsx
    Button.stories.tsx
    Button.types.ts
    index.ts

  TextField/
    TextField.tsx
    TextField.css
    TextField.test.tsx
    TextField.stories.tsx
    index.ts

  Modal/
    Modal.tsx
    Modal.css
    Modal.test.tsx
    Modal.stories.tsx
    index.ts

providers/
```

```
DesignProvider.tsx
index.ts

index.ts
```

I added providers/ because applications need a clean entry point for selecting the design label.

tokens/

Purpose

The tokens/ folder is the source of truth for design decisions as data.

It should contain:

```
foundation tokens
semantic tokens
builder tokens
token schemas
generated token artifacts
```

The Design Tokens format work recommends JSON-based token files and discusses .tokens and .tokens.json file extensions: [designtokens.org](https://designsystem.tools/guides/design-tokens/).

Contents

```
tokens/foundation/
tokens/semantic/
tokens/builder/
tokens/schema/
tokens/generated/
```

Example files

```
kanbien-core.tokens.json
color.semantic.tokens.json
button.builder.tokens.json
tokens.css
tokens.ts
```

Example foundation token

```
{
  "color": {
    "blue": {
      "600": {
        "$type": "color",
        "$value": "#1457D9"
      }
    },
    "neutral": {
      "0": {
        "$type": "color",
        "$value": "#FFFFFF"
      },
      "950": {
        "$type": "color",
        "$value": "#111827"
      }
    }
  }
},
```

```
"radius": {  
  "md": {  
    "$type": "dimension",  
    "$value": "8px"  
  }  
}
```

Allowed dependencies

Ideally none.

Acceptable:

```
JSON  
TypeScript types  
schema definitions  
generated files
```

Forbidden dependencies

```
React  
application code  
routing  
API clients  
business logic  
component imports  
CSS-in-JS runtime  
environment variables  
database access
```

Ownership rules

Owned jointly by:

```
design system lead  
brand/design lead  
front-end platform lead  
accessibility owner
```

Changes should require:

```
design review  
accessibility review for color/contrast changes  
versioned changelog  
visual regression run
```

resolvers/

Purpose

Resolvers convert token source files into usable outputs.

They answer:

```
Given design label X, what foundation tokens apply?  
How do semantic aliases resolve?  
How do builder tokens resolve?  
What CSS variables should be emitted?  
Are there cycles?  
Are contrast requirements met?
```

Are required tokens missing?

Contents

```
resolveDesignLabel.ts
resolveTokenAliases.ts
createCssVariables.ts
validateContrast.ts
validateTokenGraph.ts
buildTokens.ts
```

Example resolver

```
// packages/design-system/resolvers/resolveDesignLabel.ts

export type DesignLabel =
  | "kanbien-core"
  | "kanbien-compact"
  | "partner-acme";

export interface ResolvedDesign {
  label: DesignLabel;
  cssSelector: string;
  cssVariables: Record<string, string>;
}

export function resolveDesignLabel(label: DesignLabel): ResolvedDesign {
  // In production this would load and merge:
  // 1. foundation tokens for label
  // 2. shared semantic tokens
  // 3. shared builder tokens
  // 4. mode overrides such as light/dark/density
  return {
    label,
    cssSelector: `[data-kb-design="${label}"]`,
    cssVariables: {},
  };
}
```

Allowed dependencies

```
Node.js filesystem APIs for build scripts
token schema validation libraries
color contrast utilities
TypeScript
test runner
```

Forbidden dependencies

```
React
React DOM
application packages
component packages
browser-only APIs unless explicitly isolated
feature flags
runtime API clients
```

Ownership rules

Owned by:

```
front-end platform team
design system engineering team
```

Design should approve semantics. Engineering should own correctness, validation, generation, and build integration.

primitives/

Purpose

Primitives are low-level React building blocks.

They should be boring, stable, accessible, and domain-neutral.

Contents

```
Box
Text
Stack
Inline
Grid
Icon
VisuallyHidden
ButtonPrimitive
```

Example primitive

```
// packages/design-system/primitives/ButtonPrimitive/ButtonPrimitive.tsx

import type { ButtonHTMLAttributes, ReactNode } from "react";

export interface ButtonPrimitiveProps
  extends ButtonHTMLAttributes<HTMLButtonElement> {
  children: ReactNode;
}

export function ButtonPrimitive({
  type = "button",
  children,
  ...props
}: ButtonPrimitiveProps) {
  return (
    <button type={type} {...props}>
      {children}
    </button>
  );
}
```

This primitive uses a real `<button>`, not a clickable `<div>`, because native elements carry important accessibility behavior.

Allowed dependencies

```
React
TypeScript
generated token types
CSS variables
small utility functions
```

Possibly allowed after careful review:

```
Radix Slot-like composition utility  
class name utility
```

Forbidden dependencies

```
application code  
routing  
API calls  
auth  
analytics  
business logic  
feature flags  
domain models  
higher-level components
```

Ownership rules

Owned by the design-system engineering team.

Accessibility review should be mandatory for primitives because mistakes here spread everywhere.

patterns/

Purpose

Patterns define reusable implementation recipes and domain-neutral UX structures.

For Kanbien, I would use this folder for:

```
button style recipe  
field style recipe  
focus ring recipe  
surface recipe  
layout rhythm recipe  
empty state pattern  
loading pattern  
error pattern
```

Contents

```
patterns/button/button.pattern.ts  
patterns/button/button.pattern.css  
patterns/focusRing/focusRing.css  
patterns/field/field.pattern.ts  
patterns/surface/surface.pattern.ts
```

Example pattern

```
// packages/design-system/patterns/button/button.pattern.ts  
  
export type ButtonVariant = "primary" | "secondary" | "ghost" | "danger";  
export type ButtonSize = "sm" | "md" | "lg";  
  
export interface ButtonPatternOptions {  
  variant: ButtonVariant;  
  size: ButtonSize;  
  loading?: boolean;
```

```
}  
  
export function getButtonClassName({  
  variant,  
  size,  
  loading = false,  
}: ButtonPatternOptions): string {  
  return [  
    "kb-button",  
    `kb-button--${variant}`,  
    `kb-button--${size}`,  
    loading ? "kb-button--loading" : "",  
  ]  
    .filter(Boolean)  
    .join(" ");  
}
```

Allowed dependencies

```
primitives  
generated token contracts  
CSS variables  
small class-name utilities  
accessibility helper utilities
```

Forbidden dependencies

```
application routes  
API clients  
domain-specific copy  
business state machines  
product-specific data models
```

Ownership rules

Owned jointly by:

```
design system designers  
front-end architects  
accessibility specialists
```

Patterns are where design intent becomes repeatable implementation.

components/

Purpose

Components are the public API consumed by applications.

They should be:

```
stable  
documented  
tested  
accessible  
themeable  
versioned
```

Contents

- Button
- TextField
- Select
- Checkbox
- RadioGroup
- Modal
- Tooltip
- Tabs
- Toast
- DataTable
- Pagination

Example files

- components/Button/Button.tsx
- components/Button/Button.css
- components/Button/Button.test.tsx
- components/Button/Button.stories.tsx
- components/Button/Button.types.ts
- components/Button/index.ts

Allowed dependencies

- React
- primitives
- patterns
- generated token CSS
- accessibility libraries for complex widgets
- test utilities

For complex components like Dialog, Popover, Menu, Select, and Tooltip, using mature accessibility primitives can be a good decision. Do not hand-roll complex keyboard interaction unless the team has deep accessibility expertise.

Forbidden dependencies

- application packages
- API clients
- global app stores
- routing
- business permissions
- domain objects
- hardcoded product copy
- tenant-specific logic

Ownership rules

Owned by the design-system team.

Every component should have:

- API documentation
- usage examples
- accessibility notes
- keyboard behavior notes
- visual regression coverage
- unit tests where useful
- Storybook stories or equivalent

migration notes for breaking changes

Recommended dependency rules

Use these as architectural guardrails:

```
tokens      -> depends on nothing
resolvers   -> depends on tokens
primitives  -> depends on generated token artifacts
patterns    -> depends on primitives and generated token artifacts
components  -> depends on primitives and patterns
applications -> depends on components, patterns only where approved
```

Forbidden dependency direction:

```
tokens      <- must never import components
primitives  <- must never import components
components  <- must never import applications
design-system <- must never import product business logic
```

In monorepo terms, enforce this with lint rules or package boundary tooling.

Part 4: Evaluating the design-label architecture

You proposed:

“A design label is selected. A resolver loads a set of foundation tokens. Those generate builder tokens. Builder tokens alter primitives and patterns. Applications consume the same components but the entire visual appearance changes depending on the selected design label.”

This is a good architecture, with one important correction:

Do not let builder tokens directly mutate React components at runtime.

Instead:

```
Design label selected
  v
Token resolver resolves token graph
  v
Generated CSS variables are scoped to that design label
  v
Primitives, patterns, and components read CSS variables
  v
Same React components render with different visual appearance
```

The cleaner version is:

```
Same component API
+ different CSS variable values
= different visual system
```

Example:

```
<DesignProvider design="kanbien-core">
  <Button variant="primary">Save</Button>
</DesignProvider>

<DesignProvider design="partner-acme">
  <Button variant="primary">Save</Button>
</DesignProvider>
```

Same component. Different design label. Different appearance.

Is this a good architecture?

Yes, if the system obeys these rules:

```
Design labels select visual systems, not product behavior.
Foundation tokens are raw visual inputs.
Semantic tokens define stable UI roles.
Builder tokens define component construction.
Components consume CSS variables, not raw token files.
Most resolution happens at build time.
Runtime switching is done by changing a data attribute or class.
Accessibility validation is automated.
```

Bad version:

```
if (designLabel === "partner-acme") {
  return <SpecialAcmeButton />;
}
```

Good version:

```
[data-kb-design="partner-acme"] {
  --kb-button-primary-bg: #7c3aed;
  --kb-button-radius: 999px;
}
```

Then:

```
<Button variant="primary">Save</Button>
```

The component does not know or care which design label is active.

What large companies do that is similar

This architecture is not exotic. It is similar to how mature design systems handle themes and tokenized visual systems.

IBM Carbon says themes modify existing components to fit a visual style, and that using Carbon tokens lets developers customize components by changing universal variables rather than modifying individual components: [carbondesignsystem.com](https://carbon.design/system.com).

Atlassian documents design tokens as the single source of truth for design decisions, and its Trello guidance explains that tokens support theme compliance across light and dark modes by keeping token names stable while values differ by theme: atlassian.design.

Adobe Spectrum Web Components use an `<sp-theme>` element that provides Spectrum design tokens as CSS custom properties to everything in its DOM scope; the same docs describe variants for system, color, and scale, with token names persisting while underlying values change: opensource.adobe.com.

Salesforce's Lightning Design System guidance describes design tokens as named entities for visual attributes and recommends styling hooks so custom components can adapt to org themes and branding settings: developer.salesforce.com.

So your idea is directionally aligned with serious enterprise practice.

Potential pitfalls

Pitfall 1: Design labels become hidden product forks

A design label should not change behavior.

Bad:

```
kanbien-core Button submits form
partner-acme Button opens approval modal
```

Good:

```
kanbien-core Button is rectangular blue
partner-acme Button is rounded purple
```

Visual differences are fine. Behavioral differences should be explicit product code.

Pitfall 2: Layout shifts across design labels

If one design label makes buttons 32px tall and another makes them 52px tall, pages may break.

You need constraints:

```
min/max control heights
density modes
layout regression tests
```

documented supported variation ranges

Pitfall 3: Accessibility regressions

Every design label must pass contrast and focus visibility requirements.

You need automated checks for:

```
text contrast
button contrast
focus ring visibility
disabled-state contrast
status colors
dark mode contrast
```

Pitfall 4: Token alias cycles

Example:

```
{
  "color.action.primary.bg": "{button.primary.bg}",
  "button.primary.bg": "{color.action.primary.bg}"
}
```

This creates a loop.

Resolvers must detect cycles.

Pitfall 5: Runtime token loading creates flicker

If the application loads token JSON after the page renders, users may see a flash of unstyled or wrongly styled UI.

Prefer:

```
precompiled CSS
server-rendered design label attribute
critical CSS loaded early
```

Pitfall 6: Third-party components ignore your tokens

If Kanbien uses third-party components, they may not consume your CSS variables. Wrap or adapt them.

Do not let ungoverned third-party UI leak directly into product screens.

Pitfall 7: Too many design labels

Every new design label multiplies testing.

Before adding a design label, ask:

```
Is this a true brand/theme requirement?
Can this be solved with semantic token overrides?
Does this require new layout rules?
Who owns regression testing?
Who approves accessibility?
```

How to implement this cleanly in React

Use React for component composition.

Use CSS variables for visual theming.

Use TypeScript for safe APIs.

Use token generation for consistency.

Recommended flow:

1. Author token JSON files.
2. Validate token graph in CI.
3. Resolve aliases at build time.
4. Generate CSS variable files.
5. Generate TypeScript token names/types.
6. Import generated CSS once.
7. Use `<DesignProvider design="...">` to set a data attribute.
8. Components consume CSS variables through CSS classes.

Runtime model:

```
<DesignProvider design="kanbien-core">
  <App />
</DesignProvider>
```

Provider:

```
// packages/design-system/providers/DesignProvider.tsx

import type { ReactNode } from "react";
import "../tokens/generated/tokens.css";

export type DesignLabel =
  | "kanbien-core"
  | "kanbien-compact"
  | "partner-acme";

export interface DesignProviderProps {
  design: DesignLabel;
  children: ReactNode;
}

export function DesignProvider({ design, children }: DesignProviderProps) {
  return (
    <div data-kb-design={design}>
      {children}
    </div>
  );
}
```

Generated CSS:

```
/* packages/design-system/tokens/generated/tokens.css */

[data-kb-design="kanbien-core"] {
  --kb-color-action-primary-bg: #1457d9;
  --kb-color-action-primary-bg-hover: #0f46b8;
  --kb-color-action-primary-text: #ffffff;
  --kb-radius-control: 8px;
  --kb-size-control-md: 40px;
}

[data-kb-design="partner-acme"] {
  --kb-color-action-primary-bg: #7c3aed;
```

```
--kb-color-action-primary-bg-hover: #6d28d9;  
--kb-color-action-primary-text: #ffffff;  
--kb-radius-control: 999px;  
--kb-size-control-md: 44px;  
}
```

Component CSS:

```
.kb-button--primary {  
  background: var(--kb-button-primary-bg-default);  
  color: var(--kb-button-primary-text);  
  border-radius: var(--kb-button-radius);  
  min-height: var(--kb-button-height-md);  
}
```

The React component stays stable.

Part 5: Complete worked example - Button

Now let's trace a Button all the way through:

```
Design Label
-> Foundation Tokens
-> Semantic Tokens
-> Builder Tokens
-> Primitive
-> Pattern
-> Component
-> Application Usage
```

Step 1: Design Label

A design label is a named visual system.

```
// packages/design-system/tokens/schema/design-label.schema.ts

export type DesignLabel =
  | "kanbien-core"
  | "kanbien-compact"
  | "partner-acme";
```

Example meanings:

```
kanbien-core
Default Kanbien product visual system.

kanbien-compact
Same brand, denser enterprise layout.

partner-acme
White-label or partner-branded theme.
```

Application usage:

```
<DesignProvider design="kanbien-core">
  <App />
</DesignProvider>
```

Step 2: Foundation Tokens

Foundation tokens define raw values for a design label.

```
// packages/design-system/tokens/foundation/kanbien-core.tokens.json

{
  "color": {
    "blue": {
      "600": {
        "$type": "color",
        "$value": "#1457D9"
      },
      "700": {
        "$type": "color",
        "$value": "#0F46B8"
      }
    },
    "red": {
      "600": {
        "$type": "color",
```

```

    "$value": "#DC2626"
  },
  "700": {
    "$type": "color",
    "$value": "#B91C1C"
  }
},
"neutral": {
  "0": {
    "$type": "color",
    "$value": "#FFFFFF"
  },
  "950": {
    "$type": "color",
    "$value": "#111827"
  }
}
},
"radius": {
  "sm": {
    "$type": "dimension",
    "$value": "4px"
  },
  "md": {
    "$type": "dimension",
    "$value": "8px"
  },
  "pill": {
    "$type": "dimension",
    "$value": "999px"
  }
},
"size": {
  "control": {
    "sm": {
      "$type": "dimension",
      "$value": "32px"
    },
    "md": {
      "$type": "dimension",
      "$value": "40px"
    },
    "lg": {
      "$type": "dimension",
      "$value": "48px"
    }
  }
},
"space": {
  "2": {
    "$type": "dimension",
    "$value": "8px"
  },
  "3": {
    "$type": "dimension",
    "$value": "12px"
  },
  "4": {
    "$type": "dimension",
    "$value": "16px"
  }
}
}
}

```

Partner design label:

```
// packages/design-system/tokens/foundation/partner-acme.tokens.json

{
  "color": {
    "purple": {
      "600": {
        "$type": "color",
        "$value": "#7C3AED"
      },
      "700": {
        "$type": "color",
        "$value": "#6D28D9"
      }
    },
    "red": {
      "600": {
        "$type": "color",
        "$value": "#E11D48"
      },
      "700": {
        "$type": "color",
        "$value": "#BE123C"
      }
    },
    "neutral": {
      "0": {
        "$type": "color",
        "$value": "#FFFFFF"
      },
      "950": {
        "$type": "color",
        "$value": "#18181B"
      }
    }
  },
  "radius": {
    "sm": {
      "$type": "dimension",
      "$value": "6px"
    },
    "md": {
      "$type": "dimension",
      "$value": "14px"
    },
    "pill": {
      "$type": "dimension",
      "$value": "999px"
    }
  },
  "size": {
    "control": {
      "sm": {
        "$type": "dimension",
        "$value": "34px"
      },
      "md": {
        "$type": "dimension",
        "$value": "44px"
      },
      "lg": {
        "$type": "dimension",
        "$value": "52px"
      }
    }
  }
},
```

```

"space": {
  "2": {
    "$type": "dimension",
    "$value": "8px"
  },
  "3": {
    "$type": "dimension",
    "$value": "14px"
  },
  "4": {
    "$type": "dimension",
    "$value": "18px"
  }
}
}
}

```

At this layer, there is no Button yet.

Only raw values.

Step 3: Semantic Tokens

Semantic tokens give raw values a role.

For kanbien-core:

```

// packages/design-system/tokens/semantic/color.semantic.kanbien-core.tokens.json
{
  "color": {
    "action": {
      "primary": {
        "bg": {
          "$type": "color",
          "$value": "{color.blue.600}"
        },
        "bgHover": {
          "$type": "color",
          "$value": "{color.blue.700}"
        },
        "text": {
          "$type": "color",
          "$value": "{color.neutral.0}"
        }
      },
      "danger": {
        "bg": {
          "$type": "color",
          "$value": "{color.red.600}"
        },
        "bgHover": {
          "$type": "color",
          "$value": "{color.red.700}"
        },
        "text": {
          "$type": "color",
          "$value": "{color.neutral.0}"
        }
      }
    },
    "text": {
      "default": {
        "$type": "color",
        "$value": "{color.neutral.950}"
      }
    }
  }
}

```

```

    }
  },
  "radius": {
    "control": {
      "$type": "dimension",
      "$value": "{radius.md}"
    }
  },
  "size": {
    "control": {
      "md": {
        "$type": "dimension",
        "$value": "{size.control.md}"
      }
    }
  }
}

```

For partner-acme, the semantic token names stay the same, but values point to different foundation tokens:

```

// packages/design-system/tokens/semantic/color.semantic.partner-acme.tokens.json

{
  "color": {
    "action": {
      "primary": {
        "bg": {
          "$type": "color",
          "$value": "{color.purple.600}"
        },
        "bgHover": {
          "$type": "color",
          "$value": "{color.purple.700}"
        },
        "text": {
          "$type": "color",
          "$value": "{color.neutral.0}"
        }
      }
    }
  },
  "radius": {
    "control": {
      "$type": "dimension",
      "$value": "{radius.pill}"
    }
  }
}

```

Important: the Button still does not know about blue or purple.
It only knows about action roles.

Step 4: Builder Tokens

Builder tokens define how the Button is constructed.

```

// packages/design-system/tokens/builder/button.builder.tokens.json

{
  "button": {
    "primary": {
      "bg": {

```

```

    "default": {
      "$type": "color",
      "$value": "{color.action.primary.bg}"
    },
    "hover": {
      "$type": "color",
      "$value": "{color.action.primary.bgHover}"
    }
  },
  "text": {
    "$type": "color",
    "$value": "{color.action.primary.text}"
  }
},
"danger": {
  "bg": {
    "default": {
      "$type": "color",
      "$value": "{color.action.danger.bg}"
    },
    "hover": {
      "$type": "color",
      "$value": "{color.action.danger.bgHover}"
    }
  },
  "text": {
    "$type": "color",
    "$value": "{color.action.danger.text}"
  }
},
"radius": {
  "$type": "dimension",
  "$value": "{radius.control}"
},
"height": {
  "md": {
    "$type": "dimension",
    "$value": "{size.control.md}"
  }
},
"paddingInline": {
  "md": {
    "$type": "dimension",
    "$value": "{space.4}"
  }
}
}
}

```

The resolver turns that into CSS variables.

```

/* packages/design-system/tokens/generated/tokens.css */

[data-kb-design="kanbien-core"] {
  --kb-button-primary-bg-default: #1457d9;
  --kb-button-primary-bg-hover: #0f46b8;
  --kb-button-primary-text: #ffffff;
  --kb-button-danger-bg-default: #dc2626;
  --kb-button-danger-bg-hover: #b91c1c;
  --kb-button-danger-text: #ffffff;
  --kb-button-radius: 8px;
  --kb-button-height-md: 40px;
  --kb-button-padding-inline-md: 16px;
}

```

```
[data-kb-design="partner-acme"] {
  --kb-button-primary-bg-default: #7c3aed;
  --kb-button-primary-bg-hover: #6d28d9;
  --kb-button-primary-text: #ffffff;
  --kb-button-danger-bg-default: #e11d48;
  --kb-button-danger-bg-hover: #be123c;
  --kb-button-danger-text: #ffffff;
  --kb-button-radius: 999px;
  --kb-button-height-md: 44px;
  --kb-button-padding-inline-md: 18px;
}
```

Now the visual system can change without changing the React component.

Step 5: Primitive

The primitive owns the native HTML element and basic accessibility posture.

```
// packages/design-system/primitives/ButtonPrimitive/ButtonPrimitive.tsx

import type { ButtonHTMLAttributes, ReactNode } from "react";

export interface ButtonPrimitiveProps
  extends ButtonHTMLAttributes<HTMLButtonElement> {
  children: ReactNode;
}

export function ButtonPrimitive({
  type = "button",
  children,
  ...props
}: ButtonPrimitiveProps) {
  return (
    <button type={type} {...props}>
      {children}
    </button>
  );
}
```

This primitive should stay intentionally simple.

It should not know:

```
primary
danger
Kanbien
Acme
saving invoices
user permissions
routes
```

It only knows:

```
I render a proper HTML button.
```

Step 6: Pattern

The pattern maps component intent to implementation classes.

```
// packages/design-system/patterns/button/button.pattern.ts

export type ButtonVariant = "primary" | "secondary" | "ghost" | "danger";
export type ButtonSize = "sm" | "md" | "lg";
```

```
export interface ButtonPatternOptions {
  variant: ButtonVariant;
  size: ButtonSize;
  loading?: boolean;
  fullWidth?: boolean;
}

export function getButtonClassName({
  variant,
  size,
  loading = false,
  fullWidth = false,
}: ButtonPatternOptions): string {
  return [
    "kb-button",
    `kb-button--${variant}`,
    `kb-button--${size}`,
    loading ? "kb-button--loading" : "",
    fullWidth ? "kb-button--fullWidth" : "",
  ]
    .filter(Boolean)
    .join(" ");
}
```

Pattern CSS:

```
/* packages/design-system/patterns/button/button.pattern.css */

.kb-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;

  border: 1px solid transparent;
  border-radius: var(--kb-button-radius);

  font: inherit;
  font-weight: 600;
  cursor: pointer;

  transition:
    background-color 120ms ease,
    border-color 120ms ease,
    color 120ms ease,
    box-shadow 120ms ease;
}

.kb-button:focus-visible {
  outline: 2px solid var(--kb-color-border-focus, currentColor);
  outline-offset: 2px;
}

.kb-button:disabled {
  cursor: not-allowed;
  opacity: 0.56;
}

.kb-button--md {
  min-height: var(--kb-button-height-md);
  padding-inline: var(--kb-button-padding-inline-md);
}

.kb-button--primary {
```

```

    background: var(--kb-button-primary-bg-default);
    color: var(--kb-button-primary-text);
  }

.kb-button--primary:hover:not(:disabled) {
  background: var(--kb-button-primary-bg-hover);
}

.kb-button--danger {
  background: var(--kb-button-danger-bg-default);
  color: var(--kb-button-danger-text);
}

.kb-button--danger:hover:not(:disabled) {
  background: var(--kb-button-danger-bg-hover);
}

.kb-button--fullWidth {
  width: 100%;
}

.kb-button--loading {
  position: relative;
}

```

The pattern consumes builder tokens through CSS variables.

It does not know whether the active design is Kanbien Core or Acme.

Step 7: Component

The public Button component gives product teams a clean API.

```

// packages/design-system/components/Button/Button.tsx

import type { ButtonHTMLAttributes, ReactNode } from "react";
import { ButtonPrimitive } from "../../primitives/ButtonPrimitive";
import {
  getButtonClassName,
  type ButtonSize,
  type ButtonVariant,
} from "../../patterns/button/button.pattern";

import "../../patterns/button/button.pattern.css";

export interface ButtonProps
  extends Omit<ButtonHTMLAttributes<HTMLButtonElement>, "className"> {
  children: ReactNode;
  variant?: ButtonVariant;
  size?: ButtonSize;
  loading?: boolean;
  fullWidth?: boolean;
}

export function Button({
  children,
  variant = "primary",
  size = "md",
  loading = false,
  fullWidth = false,
  disabled,
  ...props
}: ButtonProps) {
  const className = getButtonClassName({
    variant,

```

```

    size,
    loading,
    fullWidth,
  });

  return (
    <ButtonPrimitive
      className={className}
      disabled={disabled || loading}
      aria-busy={loading || undefined}
      {...props}
    >
      {loading ? "Loading..." : children}
    </ButtonPrimitive>
  );
}

```

Export:

```

// packages/design-system/components/Button/index.ts

export { Button } from "./Button";
export type { ButtonProps } from "./Button";

```

Package entry point:

```

// packages/design-system/index.ts

export { DesignProvider } from "./providers/DesignProvider";
export type { DesignLabel } from "./providers/DesignProvider";

export { Button } from "./components/Button";
export type { ButtonProps } from "./components/Button";

```

Step 8: Application Usage

The application consumes the same component.

```

// apps/admin/src/App.tsx

import { DesignProvider, Button } from "@kanbien/design-system";

export function App() {
  return (
    <DesignProvider design="kanbien-core">
      <main>
        <h1>Project settings</h1>

        <Button variant="primary">
          Save changes
        </Button>

        <Button variant="danger">
          Delete project
        </Button>
      </main>
    </DesignProvider>
  );
}

```

For a partner-branded app:

```

// apps/partner-portal/src/App.tsx

```

```
import { DesignProvider, Button } from "@kanbien/design-system";

export function App() {
  return (
    <DesignProvider design="partner-acme">
      <main>
        <h1>Project settings</h1>

        <Button variant="primary">
          Save changes
        </Button>

        <Button variant="danger">
          Delete project
        </Button>
      </main>
    </DesignProvider>
  );
}
```

Same React code.

Different design label.

Different visual output.

```
kanbien-core:
primary button = blue, 8px radius, 40px height

partner-acme:
primary button = purple, pill radius, 44px height
```

No application code changed.

Part 6: Final recommended Kanbien architecture

For Kanbien, I would establish the design system as a package with this conceptual contract:

```
Applications express intent.  
The design system expresses UI.  
Tokens express visual decisions.  
Resolvers turn visual decisions into safe runtime artifacts.  
React components consume the artifacts.
```

The practical architecture:

```
packages/design-system/  
  tokens/      source-of-truth design data  
  resolvers/   token graph validation and generation  
  primitives/  low-level accessible building blocks  
  patterns/    reusable styling/interaction recipes  
  components/  public application-facing React components  
  providers/   design label and theme scope
```

The key rule:

```
Applications should not know how the Button is drawn.  
The Button should not know what product flow it is in.  
Tokens should not know React exists.  
React components should not hardcode brand values.
```

That separation is what gives Kanbien long-term leverage. It lets you redesign, theme, white-label, govern accessibility, and scale multiple products without each application becoming its own visual island.